
Ashen Era Archive Assistant

Team Predictrix

SLIIT Codefest 2026 – AI Competition – Sub-track 1C: Searching the Way a Human Does

Demo video: <https://youtu.be/MP3p7R12oOw>

Problem Statement and Sub-track

Some questions in the Ashen Era Archive cannot be answered by reading a single passage. The correct answer may be disputed across sources that carry different levels of trust, or it may only emerge once a reader has followed a chain of related facts scattered across several documents. For this reason we chose Sub-track 1C, Searching the Way a Human Does. Our goal was to build an assistant that searches the archive, reads what it finds, and honestly judges whether it has gathered enough evidence to answer. When it has not, it searches again using what it just learned, repeating this process until it can give a complete and properly qualified answer.

Solution Overview and Architecture



Every file in the corpus, regardless of its format or which topic folder it belongs to, is read and converted into a common internal structure. Each piece of text is also assigned a reliability score based on the type of source it came from. Codex entries are treated as official records and given a score of 0.9. Wiki pages are considered fairly dependable at 0.7. Chronicles, being narrative accounts that may reflect a character's personal bias, are scored at 0.5. Ephemera such as letters and ledgers, which the corpus itself warns can be unreliable, receive the lowest default score of 0.3.

Once every document has been read and scored, it is broken into smaller chunks and converted into embeddings, numerical representations of meaning that can be searched quickly, and these are stored in a local vector database.

The heart of the system is the agent loop. When given a question, the agent searches the archive, reads what comes back, and then makes an actual judgment call about whether it has enough information to answer or should look further. If something is missing, it searches again, guided by what it just learned, and this cycle can repeat up to five times. Before committing to a final answer, if more than one source contributed to it, a second and separate agent checks whether those sources actually agree with each other. When they do not, this conflict resolution agent decides which source to trust, giving priority to any source that explicitly claims authority over a competing account, and otherwise falling back on the default reliability scores.

Key Technical Decisions

Single sub-track, not combined with 1B. We initially considered combining 1C and 1B into one system, but the challenge document makes clear that a strong solution to a single track is valued more than a shallow attempt at several. This turned out to be the right call. During testing we found that a well built 1C loop naturally handles many 1B style multi-hop questions on its own, which we confirmed on seven real 1B sample questions, all answered correctly, without writing any separate logic for that track.

OpenRouter over Voyage AI or Grok. Voyage AI requires a credit card for identity verification even though its free tier itself costs nothing, and Grok's API has no genuine free tier at all. OpenRouter's free embedding and chat models required no card whatsoever and proved sufficient for a project of this scale.

Reliability tiering from the very beginning. We built this in early rather than adding it later, because the corpus itself gave us a clear reason to. Several codex entries state outright that they are the authoritative record and that popular accounts claiming otherwise are simply wrong. This told us that trust in a source could not be assumed equal everywhere, and needed to be something the system actively reasoned about.

Hybrid search combining meaning and exact match. A search based purely on meaning can sometimes miss an exact named entity if the wording differs even slightly from the source text. To address this, our search combines semantic similarity with an exact match check on proper noun candidates pulled directly from the question, then merges both sets of results together.

Honest uncertainty instead of a forced answer. We allow the agent to say plainly that the evidence is insufficient rather than forcing it to produce an answer no matter what. We tested this directly by asking about a detail that does not exist anywhere in the corpus. The system searched five times from different angles before honestly declining to answer, rather than inventing a plausible sounding but false response.

A second agent that only runs when needed. The conflict checking step is a separate call with one narrow task, and it only activates when two or more distinct sources were actually used to build an answer. Straightforward questions are answered without any added cost, while genuinely contested facts receive the extra scrutiny they need.

What Works, What Doesn't, and Why

Validated and working

Both official 1C sample questions were answered correctly, and in each case the system identified a genuine conflict between a codex entry and a competing ephemera source, correctly favoring the codex. Genuine multi-hop reasoning was also verified on questions we wrote ourselves rather than the official set, such as tracing a person to their faction and then to the war that faction won, and these answers required multiple real search steps rather than a single lucky guess. Honest uncertainty was confirmed as well: when asked something the corpus simply does not contain, the system said so rather than fabricating a plausible answer.

Known limitations

OCR is fully implemented but disabled by default. It proved slow on our development hardware, and the content it would unlock, mostly character portraits and creature artwork, is more relevant to sub-track 1A than to our own fact based questions, so this was a deliberate trade-off rather than something we overlooked. The system also has no dedicated architecture for sub-track 1B; its strong performance on 1B style questions is a natural side effect of a well built 1C loop rather than a separately engineered capability. Finally, OpenRouter's free tier allows fifty requests per day, which limited how much large scale testing we could run on any single day during development.

Approaches tried and abandoned

We briefly considered combining 1B and 1C into a single multi-agent system before deciding a focused 1C build would serve us better. We also initially used Pandoc to parse Word documents, but replaced it with native Python-docx parsing to remove an external dependency that caused installation problems on Windows. Finally, an early version of our document splitter assumed a single heading level would always mark the boundary between entries. Real files in the corpus nest their headings inconsistently, so we changed the approach to detect whichever heading level is actually the shallowest one present in a given file, rather than hard coding an assumption based on a single example.

AI Usage Disclosure

Claude, developed by Anthropic, was used throughout this project as an AI pair architect. It proposed and refined the overall architecture, wrote most of the implementation code, and helped diagnose real bugs using actual evidence rather than guesswork. As one example, a small diagnostic script was written and run directly against a real corpus file to correctly identify a document parsing bug, instead of attempting a blind fix.

Every important decision in this project was made by the team itself. This includes the choice of sub-track, the decision to avoid any stack that required a credit card, the choice to disable OCR, and approval of each open design question before it was implemented, including the search strategy, how uncertainty should be handled, and the design of the conflict resolution agent. Every fix made during development was tested on real hardware and reported back before it was trusted, and nothing in this system was assumed to work without being verified. The full exported conversation log is included at `ai_usage/claude.md`.

Team Members, Roles and Contributions

This team was approved by the competition organizers to register with 3 members instead of the standard 4.

Name	Role	Contribution
Shebullan Joshwel	Lead	Built the agent loop, including the search-read-decide logic and honest uncertainty handling, and the conflict resolution agent, including reliability priors and disagreement resolution. Wrote <code>architecture.md</code> , <code>decisions.md</code> , and <code>limitations.md</code> . Led end-to-end batch testing across both 1B and 1C sample questions, manually verified answers against source documents, and diagnosed real parsing bugs using targeted diagnostic scripts rather than guesswork. Compiled the AI usage disclosure and exported chat log, wrote the submission report, planned the demo video and live presentation, and handled coordination with the competition organizers.
Aqsa M A F	Member	Built the ingestion layer, including format-agnostic readers, heading-based splitting, and table extraction, and assigned reliability priors per source type in the shared document data model. Wrote the README with setup and run instructions. Validated the ingestion pipeline against the real corpus and worked through environment and dependency issues on Windows. Organized the final repository structure for submission.
Aadila Anees	Member	Built the chunking pipeline, the embedding and vector storage layer, and the hybrid search tool combining semantic and exact-match retrieval. Built the batch testing framework used to check retrieval quality across many questions at once, and designed the colored terminal output used to display live search progress and evidence sources during testing and the demo. Prepared the presentation document used for the non-live portion of the demo video.